

Software Engineering
Tutorial 1 - UML Modeling

PhD. Manh-Hung Ha

Exercise 1

We have a drawing application that manipulates geometric shapes (Circle, Rectangle, Line, ...). The class diagram used to manage the geometric shapes is shown in Figure 1. We want to export the geometric shapes through an abstract drawing interface (see the class diagram in Figure 2). The classes for exporting geometric shapes to `pdf` and `png` file formats have already been developed. We now want to export the geometric shapes to the `svg` file format. For this purpose, we have an external library for generating `svg` files whose interface does not match the expected interface (see Figure 3).

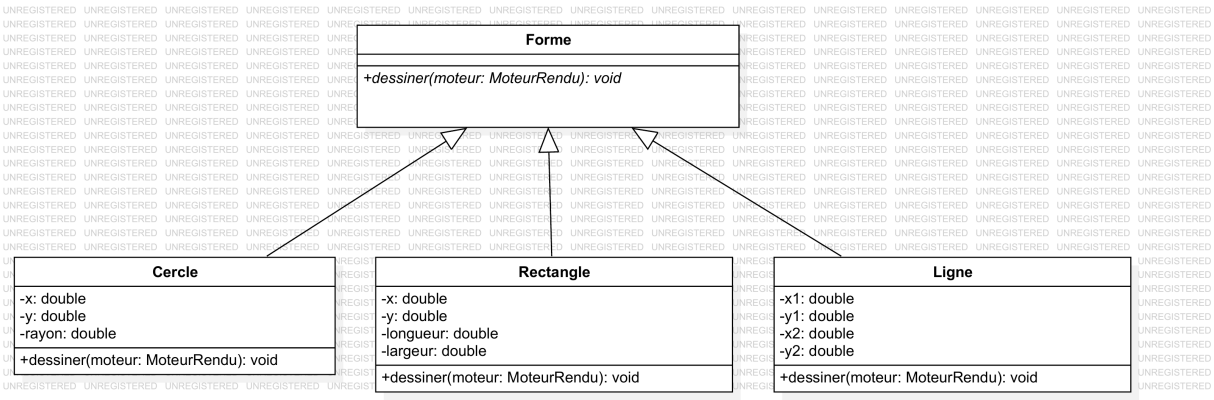


Figure 1: Class diagram for manipulating geometric shapes

Question 1 : Propose a class diagram for interfacing the `SVGWriter` class with the `MoteurRendu`, `MoteurRebduPDF` and `MoteurRenduPNG` classes. Create the class diagram using `StarUML`.

Question 2 : Provide the sequence diagram showing the messages exchanged between the objects when exporting a `Rectangle` to an `svg` file format.

We provide you with the C++ code for the classes `MoteurRendu`, `MoteurRenduPDF`, `MoteurRenduPNG`, `Forme`, `Cercle`, `Rectangle`, and `Ligne`, as well as the code for the `main` function.

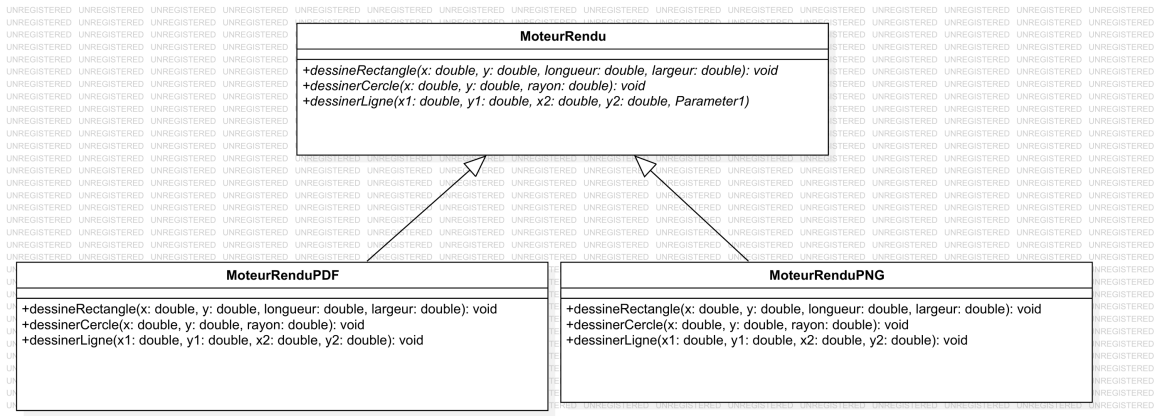


Figure 2: Class diagram for exporting geometric shapes

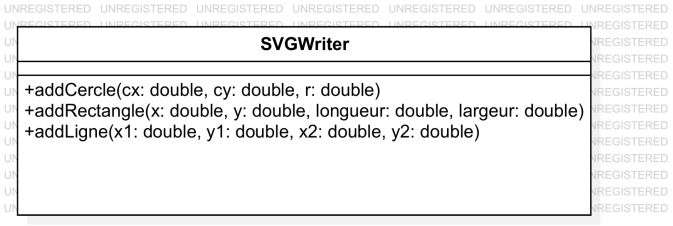


Figure 3: SVGWriter class from the external library

Question 3 : Implement the classes in your solution and complete the `main` function so that executing the `main` file produces the following output:

```
===== PDF =====
PDF : cercle
PDF : rectangle
PDF : ligne

===== PNG =====
PNG : cercle
PNG : rectangle
PNG : ligne

===== SVG =====
<circle cx="100" cy="100" r="50"/>
<rect x="20" y="30" width="200" height="100"/>
<line x1="10" y1="10" x2="20" y2="20" stroke="black"stroke-width="2"/>
```

```cpp
/**
 * @file main.cpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Main program
 *
 *
 *   g++ -std=c++20 -I. -o main main.cpp
*/
#include <iostream>

#include <moteurRenduPDF.hpp>
#include <MoteurRenduPNG.hpp>
#include <adaptateurSVG.hpp>
```

```cpp
#include <cercle.hpp>
#include <rectangle.hpp>
#include <ligne.hpp>

int main()
{
    Cercle c(100,100,50);
    Rectangle r(20,30,200,100);
    Ligne l(10, 10, 20, 20);

    MoteurRenduPNG moteurPNG;
    MoteurRenduPDF moteurPDF;
    AdaptateurSVG moteurSVG;

    std::cout << "\n===== PDF =====\n";

    c.dessiner(moteurPDF);
    r.dessiner(moteurPDF);
    l.dessiner(moteurPDF);

    std::cout << "\n===== PNG =====\n";

    c.dessiner(moteurPNG);
    r.dessiner(moteurPNG);
    l.dessiner(moteurPNG);

    std::cout << "\n===== SVG =====\n";

    c.dessiner(moteurSVG);
    r.dessiner(moteurSVG);
    l.dessiner(moteurSVG);
}
```

Exercise 2

The `Element`, `Noeud` and `Feuille` classes are given below:

```cpp
/**
 * @file element.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Abstract base class
 *
*/
class Element {
public:
    virtual void afficheElement() = 0;
    virtual ~Element() = default;
};
```

```cpp
/**
 * @file noeud.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Class used to represent a node
 *
*/
#include <iostream>
#include <vector>
#include <memory>
```

```
class Noeud : public Element {
private:
    std::vector<std::shared_ptr<Element>> lesElements;

public:
    // Constructor
    Noeud() {
        // (TO BE COMPLETED)
    }

    // Adds elements e1 and e2 to the element vector
    void setElement(std::shared_ptr<Element> e1,
                    std::shared_ptr<Element> e2) {
        // (TO BE COMPLETED)
    }

    // Method used to display the elements of a node
    void afficheElement() override {
        // (TO BE COMPLETED)
    }
};
```

```
/**
 * @file feuille.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Class used to represent a leaf
 *
*/
class Feuille : public Element {
private:
    int valeur;

public:
    // Constructor
    Feuille(int uneValeur) {
        // (TO BE COMPLETED)
    }

    // Method used to display the element of a leaf
    void afficheElement() override {
        // (TO BE COMPLETED)
    }
};
```

```
/**
 * @file main.cpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Main program
 *
 *
 *   g++ -std=c++20 -I. -o main main.cpp
*/
#include <iostream>
#include <vector>
#include <memory>
#include <element.hpp>
#include <noeud.hpp>
#include <feuille.hpp>
```

```
int main()
{
    std::shared_ptr<Noeud> n1 = std::make_shared<Noeud>();
    // TO BE COMPLETED

}
```

Question 1 : Complete the code of the `Nœud` and `Feuille` classes, as well as the code of the `main` function, in order to instantiate the object diagram shown in Figure 4.

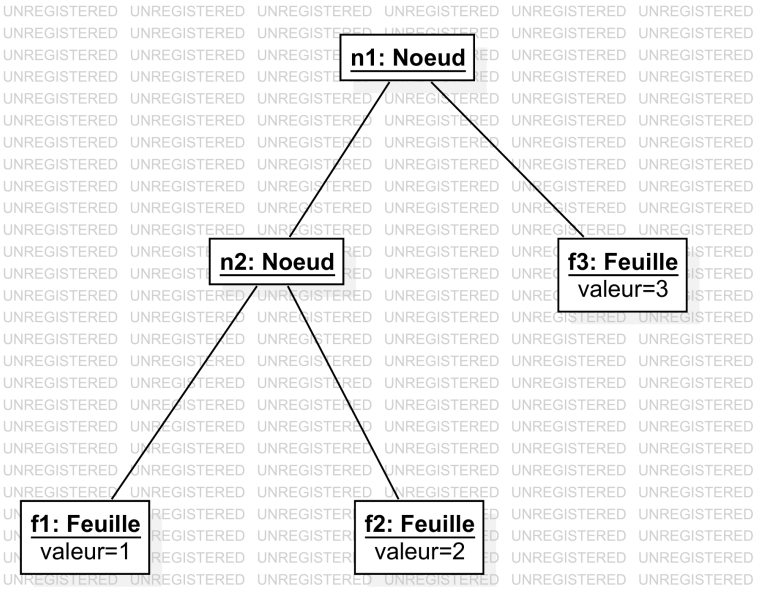


Figure 4: Diagramme d'objets

Question 2 : Provide the class diagram involving the `Element`, `Feuille` and `Nœud` classes.

Question 3 : Describe, in the form of a sequence diagram, the messages exchanged between the objects during the execution of the `afficheElement` method of object `n1`. Indicate the activation period of the objects.

Question 4 : What values are displayed on the screen when executing the `afficheElement` method of object `n2`? Indicate the type of traversal performed on the data structure being studied.

Exercise 3

The sequence diagram in Figure 5 describes the scenario of identifying a person to a system.

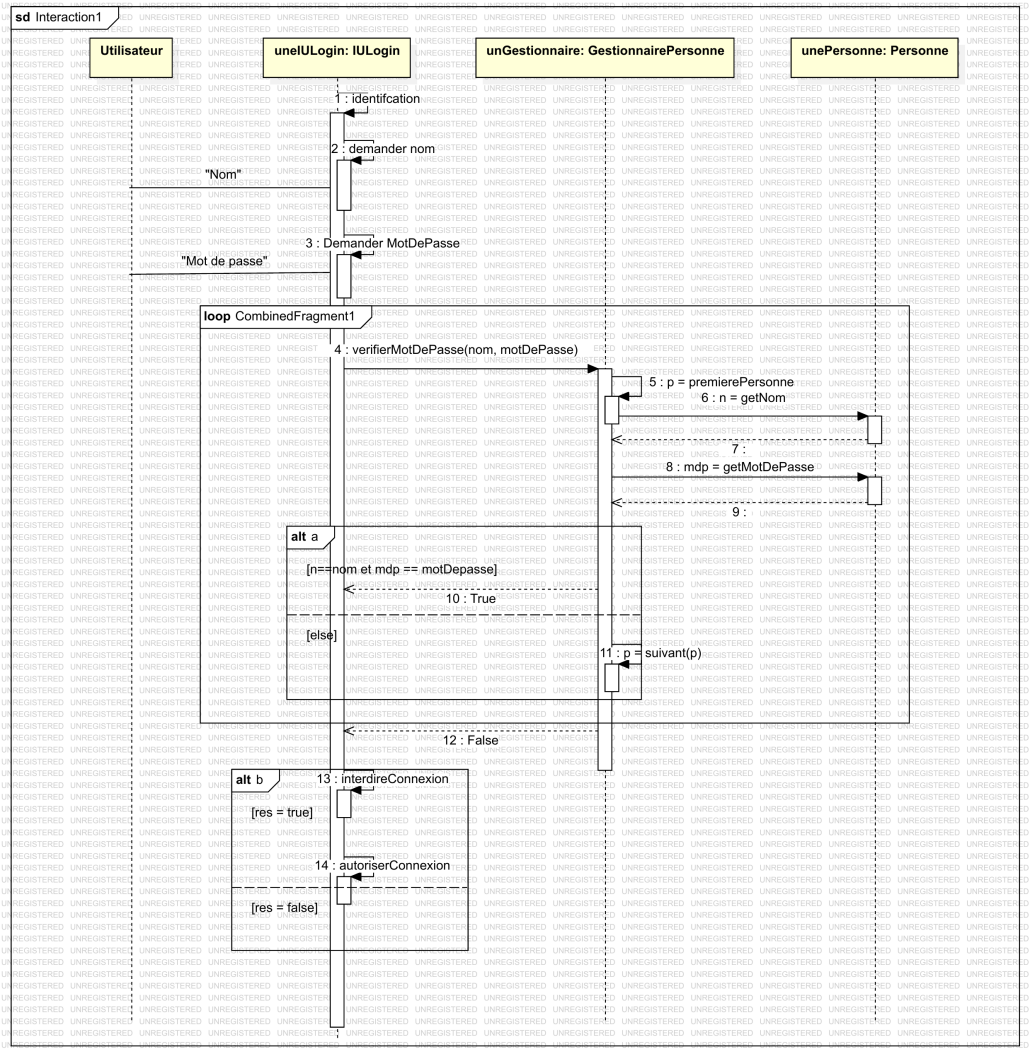


Figure 5: Sequence diagram describing the scenario of identifying a person to a system.

Question 1 : Provide the class diagram and create it using StarUML.

Question 2 : Describe the sequence of operations of the `IULogin` class using an activity diagram and create it using StarUML.

The code for the `main` function and the classes `IULogin`, `GestionnairePersonne` et `Personne` ci-dessous.

```
/**
 * @file main.cpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Main program.
 *
 * This program implements a simple authentication example.
 * It creates the different application objects, establishes the
 * associations between them, initializes a list of authorized
 * autorisées puis lance la procédure d'identification.
 *
 *   g++ -std=c++20 -I. -o main main.cpp iuLogin.cpp gestionPersonne.cpp personne.
   cpp
*/

#include <IULogin.hpp>
```

```cpp
#include <GestionnairePersonne.hpp>
#include <Personne.hpp>

int main()
{
    // Creation of the user interface responsible for communicating
    // with the user.
    std::shared_ptr<IULogin> unIULogin = std::make_shared<IULogin>();;

    // Creation of the manager that stores the people
    // autorisées à se connecter.
    std::shared_ptr<GestionnairePersonne> unGestionnaire =
                                std::make_shared<GestionnairePersonne>();

    // ------------------------------------------------------------------
    // Initialization phase
    // ------------------------------------------------------------------

    // Creation of a few users.
    // In a real application, this information would come
    // from a database or a file.
    std::shared_ptr<Personne> p1 =
                std::make_shared<Personne>("nom1", "motdepasse1");
    std::shared_ptr<Personne> p2 =
                std::make_shared<Personne>("nom2", "motdepasse2");
    std::shared_ptr<Personne> p3 =
                std::make_shared<Personne>("nom3", "motdepasse3");

    // Establishing the association between the user interface
    // and the person manager.
    unIULogin->setGestionnaire(unGestionnaire);

    // Adding the people to the manager.
    unGestionnaire->ajoute(p1);
    unGestionnaire->ajoute(p2);
    unGestionnaire->ajoute(p3);

    // ------------------------------------------------------------------
    // Application execution
    // ------------------------------------------------------------------

    // Starting the identification procedure.
    // The user is prompted to enter their name and password.
    // The manager then checks whether this information
    // matches a registered person.
    unIULogin->identification();

    return 0;
}
```

```cpp
/**
 * @file iuLogin.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Declaration of the IULogin class
 */
#pragma once

#include <string>
#include <memory>

class GestionnairePersonne;
```

```
class IULogin {
private:
    std::shared_ptr<GestionnairePersonne> gestionnaire;
    std::string nom;
    std::string motDePasse;

public:
    IULogin();
    void setGestionnaire(std::shared_ptr<GestionnairePersonne>);
    void demanderNom();
    void demanderMotDePasse();
    void autoriserConnexion();
    void interdireConnexion();
    void identification();
};
```

```
/**
 * @file iuLogin.cpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Implementation of the methods of the IULogin class
*/
#include <IULogin.hpp>
#include <GestionnairePersonne.hpp>

#include <iostream>

IULogin::IULogin() {
}

void IULogin::setGestionnaire(std::shared_ptr<GestionnairePersonne> g) {
    gestionnaire = g;
}

void IULogin::demanderNom(){
    std::cout << "Entrer le nom : ";
    std::cin >> nom;
}

void IULogin::demanderMotDePasse() {
    std::cout << "Entrer le mot de passe : ";
    std::cin >> motDePasse;
}

void IULogin::autoriserConnexion() {
    std::cout << "Connexion autorisée" << std::endl;
}

void IULogin::interdireConnexion() {
    std::cout << "Connexion interdite" << std::endl;
}

// Méthode d'identification (TO BE COMPLETED)
void IULogin::identification(){

}
```

```
/**
 * @file gestionnairePersonne.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Declaration of the GestionnairePersonne class
```

```
*/
#pragma once

#include <vector>
#include <memory>
#include <string>

#include "Personne.hpp"

class GestionnairePersonne {
private:
    std::vector<std::shared_ptr<Personne>> lesPersonnes;

public:
    GestionnairePersonne() = default;
    void ajoute(std::shared_ptr<Personne> p);
    bool verifierMotDePasse(
        const std::string& nom,
        const std::string& motDePasse) const;
};
```

```
/**
 * @file gestionnairePersonne.cpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Implementation of the methods of the GestionnairePersonne class
*/
#include <GestionnairePersonne.hpp>

void GestionnairePersonne::ajoute(std::shared_ptr<Personne> p) {
    lesPersonnes.push_back(p);
}

// Méthode verifierMotDePasse (TO BE COMPLETED)
bool verifierMotDePasse(const std::string& nom,
                        const std::string& motDePasse) const {
```

```
/**
 * @file personne.hpp
 * @author PhD. Manh-Hung Ha
 *
 * @brief Declaration of the Personne class
*/
#pragma once

#include <string>

class Personne
{
private:
    std::string nom;
    std::string motDePasse;

public:
    Personne(const std::string& nom,  const std::string& motDePasse);

    const std::string& getNom() const;
    const std::string& getMotDePasse() const;
};
```

```
/**
 * @file personne.cpp
```

```
 * @author PhD. Manh-Hung Ha
 *
 * @brief Implementation of the methods of the Personne class
*/
#include <Personne.hpp>

Personne::Personne(const std::string& nom,
                   const std::string& motDePasse)
    : nom(nom),
      motDePasse(motDePasse) {
}

const std::string& Personne::getNom() const {
    return nom;
}

const std::string& Personne::getMotDePasse() const {
    return motDePasse;
}
```

Question 3 : Complete the `identification` method of the `IULogin` class and the method `verifierMotDePasse` of the `GestionnairePersonne` class then compile the files `iuLogin.cpp`, `gestionnairePersonne.cpp` and `Personne.cpp` using the following command:

```
g++ -std=c++20 -I. -o main main.cpp iuLogin.cpp gestionPersonne.cpp personne.cpp
```

and run the `main` file.

Question 4 : Provide the object diagram showing the application objects when the `identification` method of the `IULogin` class is executed.